

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #11 - Semana 12 – Sivarama - Capitulo 20
Procesamiento de interrupciones en modo protegido

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 14/10/2024

Bibliografía:

[1] S. P. Dandamudi, *Guide to Assembly Language Programming in Linux*. Springer Science & Business Media, 2005, pp. 403–420.

Abstract—Protected-mode interrupts in the IA-32 architecture allow control flow to be altered through interrupt service routines (ISRs). Interrupts can be triggered by software or hardware, providing a mechanism for handling both planned and unplanned events. The system processes interrupts through the Interrupt Descriptor Table (IDT), which stores ISR addresses, and actions include saving flags and registers on the stack. Exceptions, categorized as faults, traps, or aborts, manage errors like division by zero. Linux system calls use software interrupts for file I/O operations, while hardware interrupts allow external devices to request CPU attention. Direct I/O device access is achieved through assembly instructions like *in* and *out*.

I. Interrupciones en modo protegido

Las interrupciones son mecanismos que alteran el flujo de control de un programa, similares a las llamadas a procedimientos. Sin embargo, las interrupciones pueden ser provocadas tanto por hardware como por software. Las rutinas de servicio de interrupciones (ISR) se encargan de manejar estos eventos, y después de ejecutarlas, el programa interrumpido retoma su ejecución. Este mecanismo permite responder eficientemente a eventos externos no planificados, como errores o acciones del usuario (por ejemplo, pulsar Ctrl-C para interrumpir un programa).

II. Clasificación de las interrupciones

Las interrupciones pueden clasificarse en tres tipos principales:

- **Interrupciones de software:** provocadas mediante la instrucción `int`, utilizadas para acceder a dispositivos como el teclado o la pantalla. Pueden ser definidas por el sistema o por el usuario.

- **Interrupciones de hardware:** generadas por dispositivos externos que solicitan atención del procesador. Pueden ser enmascarables (pueden retrasarse) o no enmascarables (atendidas inmediatamente).
- **Excepciones:** manejan errores durante la ejecución de instrucciones, como la división entre cero. Estas se dividen en faltas, trampas y abortos, dependiendo del tipo de error y si la instrucción se reinicia después de la excepción.

III. Procesamiento de interrupciones en modo protegido

En la arquitectura IA-32, las interrupciones se identifican mediante un número de tipo (de 0 a 255). Cada número corresponde a una entrada en la tabla de descriptores de interrupción (IDT), que contiene direcciones de las ISR. Cuando ocurre una interrupción, el procesador realiza las siguientes acciones:

1. Guarda el registro EFLAGS en la pila.
2. Limpia las banderas de interrupción y trampa.
3. Guarda los registros CS y EIP en la pila.
4. Carga CS con el selector de segmento desde la puerta de interrupción.
5. Carga EIP con el desplazamiento de 32 bits desde la puerta de interrupción.

El retorno de una ISR se realiza mediante la instrucción `iret`, que restaura los valores de EIP, CS y EFLAGS desde la pila.

IV. Excepciones

Las excepciones se dividen en:

- **Faltas:** errores detectados antes de ejecutar una instrucción, como la división por cero. El sistema restaura el estado anterior para que la instrucción pueda reiniciarse.
- **Trampas:** errores detectados después de ejecutar una instrucción, como el desbordamiento. No reinician la instrucción.
- **Abortos:** reportan errores graves, como fallos de hardware.

El código de error asociado con algunas excepciones se coloca en la pila, permitiendo que el manejador de excepciones lo use para diagnosticar el problema.

V. Llamadas al sistema de Linux para entrada/salida de archivos

En Linux, las interrupciones de software permiten acceder a dispositivos de entrada/salida mediante llamadas al sistema. Los archivos, el teclado y la pantalla son tratados como flujos de bytes. Algunas llamadas importantes son:

- **Crear archivo (syscall 8):** Crea un nuevo archivo. Se pasan como parámetros el nombre del archivo (EBX) y los permisos (ECX). Devuelve un descriptor de archivo en EAX o un código de error.
- **Abrir archivo (syscall 5):** Abre un archivo existente. Los parámetros incluyen el nombre del archivo (EBX), modo de acceso (ECX) y permisos (EDX). Devuelve un descriptor de archivo o un código de error.
- **Leer archivo (syscall 3):** Lee datos de un archivo abierto, especificando el descriptor del archivo (EBX), el búfer de entrada (ECX) y el tamaño del búfer (EDX). Devuelve el número de bytes leídos o un código de error.
- **Escribir archivo (syscall 4):** Escribe datos en un archivo abierto, utilizando parámetros similares a los de la lectura.
- **Cerrar archivo (syscall 6):** Cierra un archivo abierto utilizando el descriptor del archivo.
- **Mover el puntero de archivo (syscall 19):** Cambia la posición del puntero en un archivo abierto, permitiendo el acceso aleatorio a los datos.

VI. Interrupciones de hardware

Las interrupciones de hardware son generadas por dispositivos externos, como el teclado o el disco duro, solicitando atención del procesador. Estas pueden ser enmascarables o no enmascarables. Las interrupciones de hardware utilizan un controlador de interrupciones programable (APIC) para gestionar múltiples dispositivos y priorizar las solicitudes de interrupción. El procesador reconoce estas interrupciones mediante una secuencia de reconocimiento y el dispositivo interrumpe enviando un vector de interrupción al procesador.

VII. Control directo de dispositivos de entrada/salida

El acceso directo a dispositivos de entrada/salida se realiza mediante instrucciones de I/O en el ensamblador. Las instrucciones in y out se utilizan para transferir datos entre un puerto de I/O y un registro. Estos puertos están mapeados en un espacio de direcciones de 64K y pueden transferir datos de 8, 16 o 32 bits. Para transferencias masivas, se utilizan las instrucciones ins y outs, que permiten mover bloques de datos entre la memoria y los puertos de I/O.